A PROJECT REPORT ON

Smart Energy Meter

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRICAL ENGINEERING

Submitted by

1.Piyush Uday Gaikwad (2019BTEEL00067)

WALCHAND COLLEGE OF ENGINEERING, SANGLI

2024-25

WALCHAND COLLEGE OF ENGINEERING,

SANGLI

ABSTRACT

This project entails the design and implementation of an Energy Meter system based on IoT that can measure voltage, current, power, power factor, and energy consumption. The system uses ZMPT101B voltage sensor and ACS712-30A current sensor connected to an Arduino microcontroller.

A phase shift-based method is employed for the calculation of power factor. The treated electrical parameters are transmitted to Adafruit IO via a Python script on an attached PC for remote real-time monitoring and logging of energy usage.

The system offers a cost-effective and scalable personal and institutional energy auditing solution that facilitates improved awareness and control of power consumption. Enhancements in the future could be wireless integration, cloud computing, and mobile application interfaces.

TABLE OF CONTENTS

CERTIFICATE……3

DECLARATION…… 4

ABSTRACT……5

TABLE OF CONTENTS …….……..6

1.Introduction……..7

2.Problem Statement ……..8

3.Problem solving method……..9

4.Equipments……...……..11

5.Arduino code flow diagram……....12

6.Connection diagram……...16

7.Working and mathematical Modelling...……...17

8. Results and discussions..…….…… 23

9.Conclusion and References ……..…….26

1.Introduction

1.1 Need to Measure Electrical Parameters Accurate measurement of electrical parameters such as voltage, current, power, and energy is essential for the effective monitoring, operation, and safety of electrical systems. In both domestic and industrial setups, understanding power consumption patterns allows users to identify inefficiencies, perform troubleshooting, and ensure compliance with safety standards. It helps in assessing equipment performance, identifying overloads, and optimizing energy usage. Real-time and historical data of these parameters are critical for energy audits, preventive maintenance, and power quality analysis.

1.2 Measurement Techniques Voltage and current measurement techniques have evolved with the advancement of electronics. Traditional tools such as digital multimeters are still widely used for manual measurements, while modern sensor-based systems provide continuous monitoring. Sensors like ZMPT101B for voltage and ACS712 for current are popular due to their ease of integration with microcontrollers like Arduino. These sensors output analog signals corresponding to voltage and current levels, which are further processed to calculate power and energy.

1.3 Importance of Monitoring and Visualization Monitoring electrical parameters in real time provides significant advantages such as immediate detection of faults, efficient energy usage, and the ability to perform predictive maintenance. Visualization tools, especially when integrated with IoT platforms, allow for a graphical interpretation of data, enabling users to make informed decisions. Dashboards displaying live values and historical trends help in identifying anomalies, peak loads, and areas of energy wastage.

1.4 IoT in Electrical Systems The Internet of Things (IoT) has transformed traditional energy meters into smart systems capable of data logging, remote monitoring, and cloud-based analytics. By leveraging IoT platforms like Adafruit IO, sensor data from the energy meter can be transmitted to the internet using microcontrollers and WiFi modules. This integration allows users to track energy consumption from anywhere, receive alerts, and even control appliances remotely. IoT-based energy meters play a crucial role in building smart homes, industries, and cities with improved energy management and sustainability.

2. Problem Statement

Traditional energy metering systems lack the capability to provide real-time consumption data, remote accessibility, and advanced analytics, limiting users' ability to efficiently monitor and control their energy usage. As energy demand rises and smart grid infrastructure expands, there is a critical need for an IoT-enabled smart energy meter that offers accurate, real-time data acquisition, remote monitoring, and seamless integration with cloud platforms for advanced data analysis and energy management. Such a system can enhance energy efficiency, enable dynamic pricing, and support demand-side management in both residential and industrial environments.

3.Problem solving method

3.1 Sensor Choice

The ZMPT101B is selected for voltage sensing because it is precise and has isolation. The ACS712-30A sensor is utilized to measure current, with its analog output being proportional to the current through it. The sensors are connected to the analog input pins of an Arduino Uno.

3.2 Arduino Integration

The Arduino Uno is the central data acquisition board. It samples analog voltage and current from the sensors, converts them to voltage and current, and calculates power, power factor, and energy. The processed data is formatted and transmitted through the serial port to the host computer.

3.3 Signal Calibration and Conditioning

Raw analog values are calibrated with known voltage and current sources. This calibration is done to ensure that the readings are accurate. Noise filtering and offset elimination are done in the Arduino code to improve the quality of the measurements.

3.4 Data Processing in Arduino

The Arduino measures RMS voltage and current from multiple readings across a cycle. The actual power is computed using instantaneous values, while the power factor is approximated by the phase difference between the voltage and current signals. Energy is summed up by integrating over power in time.

3.5 Python Script for Adafruit IO

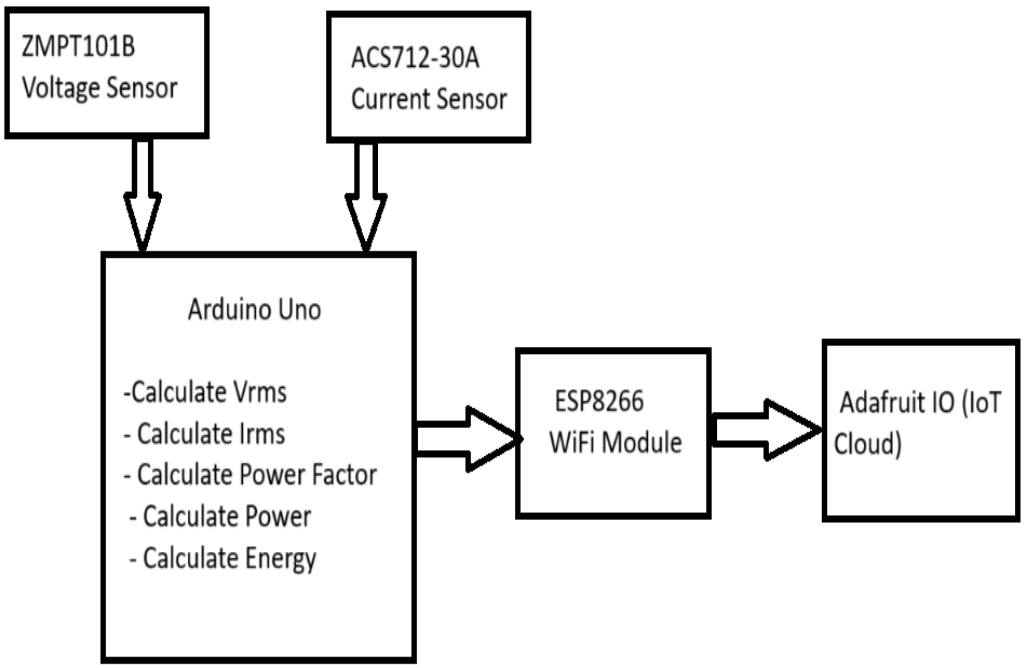
A Python script executes on a connected machine, which reads serial data from the Arduino. The script interprets the data, formats it in the correct way, and uploads it to Adafruit IO via HTTP requests. The script utilizes libraries such as serial, requests, and time for execution.

3.6 Flow Diagram of System

The flow process includes:

Sensor data acquisition → Arduino processing → Serial transmission → Python parsing → HTTP POST → Adafruit IO visualization

3.7 Block diagram

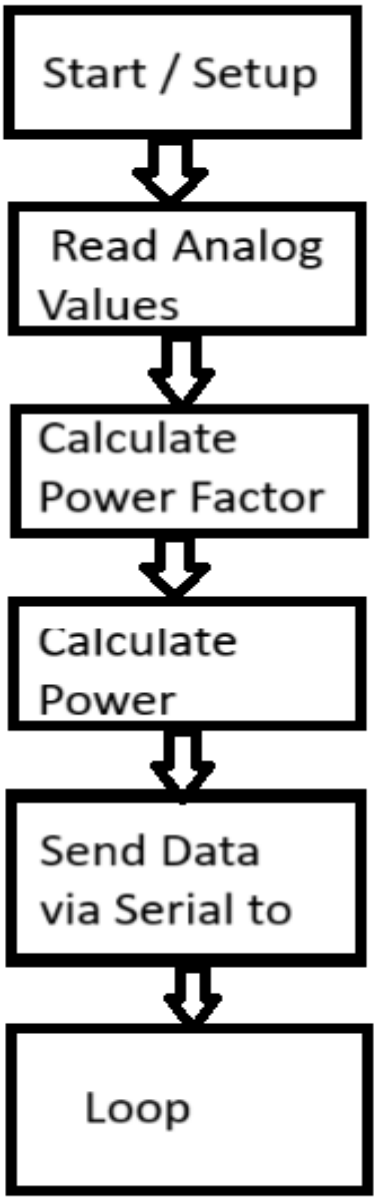


4.Equipments

Equipments	Specifications
Arduino uno R3	ATmega328P, 6 analog inputs
ZMPT101B	AC voltage sensor, up to 250V
ACS712-30A	AC/DC current sensor, 30A max
Python	Serial & HTTP data push to Adafruit
Adafruit IO	Cloud dashboard for visualization

Breadboard
Jumper wires
AC source(Dimmer,250v)
1.5 mm connecting wires

5.Ardiuno uno code flowchart



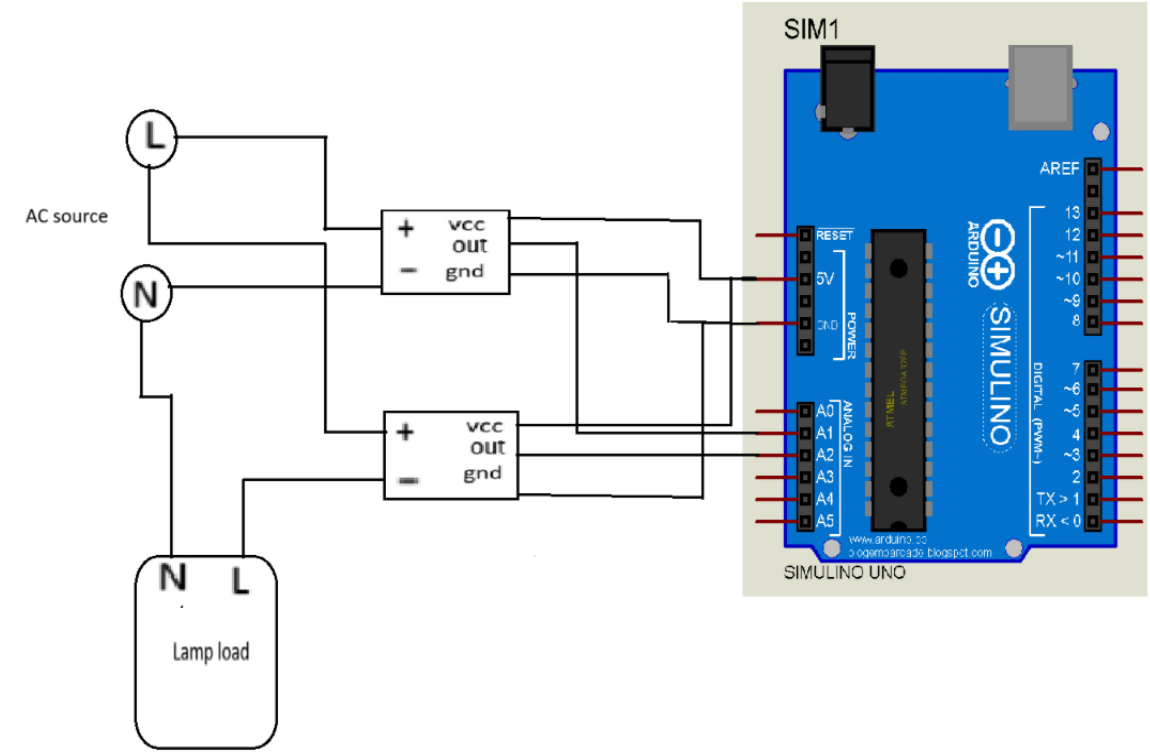
Arduino code

```
const int voltagePin = A1;            // ZMPT101B voltage sensor
const int currentPin = A2;            // ACS712 current sensor
const float VREF = 5000.0;            // 5V in millivolts
const float ADC_RES = 1023.0;         // 10-bit ADC resolution
const float OFFSET = VREF / 2.0;      // Sensor mid-point offset
const float SENSITIVITY = 92.0;       // mV per Amp for ACS712-30A
const float VOLTAGE_CALIBRATION = 0.5; // Adjust this value based on testing
const float FREQ = 50.0;              // Mains frequency in Hz
const float CYCLE_TIME = 1000.0 / FREQ; // 20 ms for 50 Hz

// === Energy Calculation Variables ===
float totalEnergy_Wh = 0.0;
unsigned long lastEnergyCalcTime = 0;

void setup() {
  Serial.begin(9600);
  lastEnergyCalcTime = millis();
}

void loop() {
  const int NUM_SAMPLES = 1000;
  float sumVoltageSq = 0;
  float sumCurrentSq = 0;

  for (int i = 0; i < NUM_SAMPLES; i++) {
    int rawV = analogRead(voltagePin);
```

```
    int rawI = analogRead(currentPin);

    float voltage = ((rawV * VREF) / ADC_RES) - OFFSET;
    float current = (((rawI * VREF) / ADC_RES) - OFFSET) / SENSITIVITY;

    sumVoltageSq += voltage * voltage;
    sumCurrentSq += current * current;
  }

  float Vrms = sqrt(sumVoltageSq / NUM_SAMPLES) * VOLTAGE_CALIBRATION;
  float Irms = sqrt(sumCurrentSq / NUM_SAMPLES);

  // Phase difference and Power Factor
  unsigned long zeroTimeV = waitForZeroCrossing(voltagePin);
  unsigned long zeroTimeI = waitForZeroCrossing(currentPin);
  long timeDiff = zeroTimeI - zeroTimeV;

  float phaseDeg = ((float)timeDiff / (CYCLE_TIME * 1000.0)) * 360.0;
  if (phaseDeg > 180.0) phaseDeg -= 360.0;
  float phaseRad = radians(phaseDeg);
  float powerFactor = cos(phaseRad);

  float power = Vrms * Irms * powerFactor;
  unsigned long now = millis();
  float deltaTimeHours = (now - lastEnergyCalcTime) / 3600000.0;
  totalEnergy_Wh += power * deltaTimeHours;
  lastEnergyCalcTime = now;
```

```
  Serial.print("Vrms: "); Serial.print(Vrms, 2); Serial.print(" V | ");
  Serial.print("Irms: "); Serial.print(Irms, 2); Serial.print(" A | ");
  Serial.print("PF: "); Serial.print(powerFactor, 3); Serial.print(" | ");
  Serial.print("Power: "); Serial.print(power, 2); Serial.print(" W | ");
  Serial.print("Energy: "); Serial.print(totalEnergy_Wh, 4); Serial.println(" Wh");

  delay(1000);
}

// === Zero Crossing Detection ===
unsigned long waitForZeroCrossing(int pin) {
  float prev = analogRead(pin) * VREF / ADC_RES;
  while (true) {
    float sample = analogRead(pin) * VREF / ADC_RES;
    if (prev < OFFSET && sample >= OFFSET) {
      return micros();
    }
    prev = sample;
  }
}
```

6. Connection diagram



7.Working and Mathematical Modelling

7.1 Voltage Measurement Using ZMPT101B

The ZMPT101B is a voltage sensor, specifically an analog voltage sensor used for AC voltage measurement. It has a precision transformer and conditioning circuitry.

Features:

- Voltage measurement up to 250V AC
- High accuracy and insulation
- Onboard adjustable gain through potentiometer

Working:

- ZMPT101B gives an analog waveform that is proportional to the input AC voltage.
- Arduino reads this waveform and computes Vrms using peak-to-peak technique.
- Signal is conditioned by an op-amp for precision and sent to A1 pin on Arduino Uno.

Formula:

Voltage = (Vout x (5/1024) -Voff) x Callibration

Vrms = Voltage/samples

• Voltage up to 250 volts can be measured

• Light weight with on-board micro-precision voltage transformer

• High precision on-board op-amp circuit

• Operating temperature: 40ºC ~ + 70ºC

• Supply voltage 5 volts to 30 volts

7.2 Measurement Using ACS712-30A

ACS712 is a Hall effect current sensor, giving an analog voltage that is centered around 2.5V at zero current.

Specifications:

- Sensitivity: 92 mV/A (for 30A version)
- Bidirectional current sensing
- Isolation from high-voltage circuits

Operation:

- The output varies with the AC current flowing through the sensor.
- Arduino picks up this signal through analog pin A2 and calculates

Formula:

Voltage = (Vout x (5/1024) -Voff)

Irms = voltage x sensitivity

Features:

• 80kHz bandwidth

• 66 to 185 mV/A output sensitivity

• Low-noise analog signal path

• Device bandwidth is set via the new FILTER pin

• 1.2 mΩ internal conductor resistance

• Total output error of 1.5% at TA = 25°C

• Stable output offset voltage.

• Near zero magnetic hysteresis

7.3 Arduino uno R3 (microcontroller unit)

The Arduino Uno is an open-source microcontroller board that is built upon the ATmega328P microcontroller. It acts as the master controller for the reception of sensor data, signal processing, and interfacing with the IoT system through serial communication

Feature	**Value**
Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7–12V
Analog Input Pins	6 (A0 to A5)
Digital I/O Pins	14 (of which 6 provide PWM)
Flash Memory	32 KB (0.5 KB used by bootloader)
SRAM	2 KB

Feature	Value
EEPROM	1 KB
Clock Speed	16 MHz
ADC Resolution	10-bit (0–1023 values)
USB Connectivity	Type-B port (Serial data transfer)
Communication Protocols	UART, SPI, I2C

Use in This Project:

- Analog Inputs A1 and A2 are used to sample the voltage (ZMPT101B) and current (ACS712-30A) sensors' values.
- It does real-time signal sampling, zero-cross detection, and power factor calculation.
- Outputs a nice formatted string through Serial, with all the measured parameters.
- Interfaces with ESP8266 (optional) or a Python script through USB for IoT integration.
- The Arduino Uno was selected for simplicity, reliability, community presence, and sufficient analog resolution (10-bit) for adequate signal monitoring and RMS computations.

7.4 Calculation of Power Factor Using Phase Shift

To calculate the power factor, the voltage and current waveform phase angle is measured.

Method:

- Zero-cross detection is performed in Arduino.
- Time delay (Δt) between voltage and current zero-crossings is computed.
- Phase angle $\phi=(\Delta t/T) \times 360$
- Power factor = $\cos(\phi)$

7.5 Power and Energy Calculation

- Active Power (P) = Vrms x Irms x Power Factor
- Energy (Wh) = Power x Time (hrs)
- Arduino calculates cumulative energy through millis() function to monitor time between readings.

7.6 Data Transmission to Adafruit IO

Rather than employing the ESP8266 to send HTTP requests directly (for saving memory and simplicity), the Arduino sends pre-formatted serial strings over USB to a plugged-in PC.

Python Script Functions:

- Listens to serial port (e.g., COM3)
- Parses data in format:[PF: 0.95 | Power: 196.45 W | Energy: 1.52 Wh]
- Uses request library to push data to Adafruit IO

8.Results

8.1 Hardware Results

Arduino uno		Multimeter		Power Factor	Power
Voltage	Current	Voltage	Current		
40	0.38	38.57	0.46	0.875	13.3
100.6	0.6	95.78	0.7	0.874	52.75
150	3.07	96.78	3.32	0.876	436.25
200	3.6	192.87	3.78	0.898	646.56
230	3.92	221.09	4.06	0.899	810.54

8.2 Software Results

My Feeds

Feed Name	Last Value
energy	0.0002
power	2.07
power factor	0.439

My Feeds

Feed Name	Last Value
energy	0.0098
power	99.15
power factor	0.848

My Feeds

Feed Name	Last Value
energy	0.7545
power	137.65
power factor	0.870

8.3Future Improvements

- Add a relay control mechanism for remote load management through Adafruit IO.
- Add over-voltage or over-current notifications through email or mobile alert.
- Implement offline data logging through onboard memory (e.g., EEPROM or SD card).
- Upgrade to ESP32 to enable direct Wi-Fi connectivity without an external PC or script.

9. Conclusion and Future scope

9.1 Conclusion

This project effectively illustrates the design and implementation of a low-cost real-time energy monitoring system with Arduino and IoT integration. Interfacing the ZMPT101B voltage sensor and ACS712-30A current sensor with an Arduino Uno enables the system to measure electrical parameters like voltage, current, real power, power factor, and energy consumption accurately.

The processed data is sent over serial communication to a Python client, which sends the values to Adafruit IO(a cloud-based service for real-time visualization). This arrangement gives the user a scalable and easy-to-access dashboard that can be remotely accessed, giving users constant access to their energy consumption patterns.

The project also incorporates necessary power factor calculations via phase shift detection to enhance measurement accuracy and value addition compared to simple energy meters. Hardware test results and software validation results validate the system's accuracy and responsiveness under changing load conditions.

The method employed is modular, inexpensive, and adaptable and is therefore ideally suited for further development in smart metering, home automation, and industrial energy analytics.

9.2 Future scope

1. Relay Control: Add IoT-controlled relays to remotely switch appliances on/off according to load conditions or time schedules.

2. Alert System: Add SMS, email, or app-based alerts for over-voltage, high current consumption, or high energy consumption.

3. Offline Logging: Include SD card support or EEPROM use to log data during network outages for syncing later.

4. Wi-Fi Integration using ESP32: Swap Arduino and external WiFi modules with an ESP32 for a more wireless, compact design.

5. Mobile App Dashboard: Develop a mobile app to display the Adafruit IO feeds in a simple, custom interface.

6. Billing System: Add tariff-based energy cost estimation to calculate electricity bills in real-time.

7. Solar Compatibility: Take the project further to measure solar panel output, inverter efficiency, and net metering.

9.3 Reference

1. https://www.edn.com/voltage-sensor/
2. https://www.allegromicro.com/-/media/files/datasheets/acs712-datasheet.ashx
3. https://components101.com/microcontrollers/arduino-uno
4. https://io.adafruit.com/
5. https://ieeexplore.ieee.org/document/9566454